Build Task — Graph Skeleton (feeds into Project 3) — Hints & Solution

> Back to the Build Task · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried.

- Hint 1 — What you're building, and the exact pieces
- Hint 2 — The plan, and almost the whole thing
- Solution

> Back to the Build Task · Hint 1 · Hint 2 · Solution

Hint 1 — What you're building, and the exact pieces

How the risky tool is caught:

- `think` looks at the model's `tool_calls`. No calls → `next_action = "finish"`. Calls → `"call_tool"`. Any call to a tool in `RISKY_TOOLS` → `"risky"`.
- `route` returns `state["next_action"]`; the edge maps `call_tool → act`, `risky → risky_action`, `finish → END`.
- `risky_action` calls `interrupt({"question": ...})`. On `Command(resume=True)` it runs the calls with the same helper as `act`; on `resume=False` it answers every call with a "Not approved" tool message — the model must get an answer for every call it made (Doc07).
- Both `act` and `risky_action` go back to `think`, so the model writes the final answer.

Then the parts the Intermediate exercises practiced:

- **No swallowed errors:** `run_tool()` already turns a tool failure into an error string; also log it with `get_logger()`, and keep it in `steps`.
- **A loud failure on a bad route:** the Edge cases exercise's named error — `route()` raises `UnhandledRouteError` for any `next_action` it doesn't handle.
- **`build_graph(checkpointer=None)`:** the caller chooses the checkpointer.
- **A real restart:** the Failure exercise's `SqliteSaver` and `graph.get_state(config)` — one script pauses and exits, a second, new process checks what's paused and resumes it.

> Back to the Build Task · Hint 1 · Hint 2 · Solution

Hint 2 — The plan, and almost the whole thing

```
exceptions.py: Doc07's file + UnhandledRouteError
nodes.py:      run_calls() also logs (info on success, warning on error)
               route() raises UnhandledRouteError if next_action is not
               one of HANDLED_ROUTES
graph.py:      build_graph(checkpointer=None) -> MemorySaver() if None
test_graph.py: 5 cases, each printing OK or WRONG
               (no risky action, pauses, approved, rejected, unknown route)
pause_demo.py:  SqliteSaver -> invoke the refund task -> print the question
resume_demo.py: a NEW process: SqliteSaver -> graph.get_state(config)
                -> if .next is set: Command(resume=True)
```

Run `python pause_demo.py`, let it exit, then `python resume_demo.py`. If the second script finishes the refund, state really survived a restart — the README's "from a fresh process" Test Case.

> Back to the Build Task · Hint 1 · Hint 2 · Solution

Solution

Story — `exceptions.py` + `nodes.py`: `run_calls()` now logs every tool result and every failure, and `route()` raises the Edge cases exercise's `UnhandledRouteError` for a `next_action` it doesn't handle. **If not:** a failed tool would only be visible if you read the state, and a typo in a `next_action` value would be a confusing `KeyError` from inside LangGraph.

```
# exceptions.py — the class added at the end of Doc07's file
class UnhandledRouteError(Exception):
    """Raised when route() gets a next_action the graph doesn't handle."""
```

```
# nodes.py
from langchain_openai import ChatOpenAI
from langgraph.types import interrupt

from agent import AGENT_TOOLS, run_tool
from exceptions import UnhandledRouteError
```

```
from logging_setup import get_logger
from refunds import issue_refund
from state import AgentState

ALL_TOOLS = AGENT_TOOLS + [issue_refund]
RISKY_TOOLS = ["issue_refund"]
HANDLED_ROUTES = ["call_tool", "risky", "finish"]

def think(state: AgentState) -> dict:
    model = ChatOpenAI(model="gpt-4o-mini", temperature=0)
    response = model.bind_tools(ALL_TOOLS).invoke(state["messages"])
    next_action = "finish"
    if response.tool_calls:
        next_action = "call_tool"
    for call in response.tool_calls:
        if call["name"] in RISKY_TOOLS:
            next_action = "risky"
    return {"messages": [response], "next_action": next_action}

def run_calls(calls: list[dict]) -> dict:
    logger = get_logger(__name__)
    tool_messages = []
    steps = []
    for call in calls:
        result, error = run_tool(call, ALL_TOOLS)
        observation = result
        if error is None:
            logger.info(f"{call['name']} -> {result}")
        else:
            # why: never swallowed -- it is logged here AND kept in state
            observation = f"Error: {error}"
            logger.warning(f"{call['name']} failed: {error}")
        tool_messages.append({
            "role": "tool",
            "content": observation,
            "tool_call_id": call["id"],
        })
        steps.append({
            "tool": call["name"],
            "arguments": call["args"],
            "result": result,
            "error": error,
        })
    return {"messages": tool_messages, "steps": steps}

def act(state: AgentState) -> dict:
    return run_calls(state["messages"][-1].tool_calls)
```

```
def risky_action(state: AgentState) -> dict:
    calls = state["messages"][-1].tool_calls
    first = calls[0]
    question = f"Approve {first['name']} with {first['args']}?"
    approved = interrupt({"question": question})
    if approved:
        update = run_calls(calls)
        update["approved"] = True
        return update

    tool_messages = []
    for call in calls:
        tool_messages.append({
            "role": "tool",
            "content": "Not approved by a human. Do not try again.",
            "tool_call_id": call["id"],
        })
    return {"messages": tool_messages, "approved": False}

def route(state: AgentState) -> str:
    action = state["next_action"]
    if action not in HANDLED_ROUTES:
        raise UnhandledRouteError(
            f"route() got next_action={action!r}; "
            f"it only handles {HANDLED_ROUTES}"
        )
    return action
```

Story — `test_graph.py` (Intermediate): 5 cases, one function each, printing `OK` or `WRONG` — including the **rejection** path and a bad route. **If not:** "a rejected refund never runs" would be untested, and it's the most important promise a human-approval step makes.

```
# test_graph.py
# Runs the Test Cases from the README. Each case prints OK or WRONG.
# Run it from inside this folder: python test_graph.py
# Makes real API calls — needs a valid OPENAI_API_KEY in .env.
from langgraph.checkpoint.memory import MemorySaver
from langgraph.types import Command

from exceptions import UnhandledRouteError
from graph import build_graph
from nodes import route

def start(task: str) -> dict:
    return {
```

```
        "messages": [{"role": "user", "content": task}],
        "steps": [],
        "next_action": "",
        "approved": False,
    }

def config_for(thread_id: str) -> dict:
    return {"configurable": {"thread_id": thread_id}}

def tools_used(state: dict) -> list[str]:
    names = []
    for step in state["steps"]:
        names.append(step["tool"])
    return names

def test_no_risky_action(graph) -> None:
    final = graph.invoke(start("What is 12 + 30?"), config=config_for("t1"))
    result = "OK"
    if "__interrupt__" in final or "add" not in tools_used(final):
        result = "WRONG"
    print(f"case 1 (no risky action) -> {result}")

def test_risky_action_pauses(graph) -> None:
    task = start("Please issue a refund for order A100.")
    paused = graph.invoke(task, config=config_for("t2"))
    result = "OK"
    if "__interrupt__" not in paused:
        result = "WRONG"
    print(f"case 2 (pauses)          -> {result}")

def test_approved_resume(graph) -> None:
    final = graph.invoke(Command(resume=True), config=config_for("t2"))
    result = "OK"
    if not final["approved"] or "issue_refund" not in tools_used(final):
        result = "WRONG"
    print(f"case 3 (approved)        -> {result}")

def test_rejected_resume(graph) -> None:
    task = start("Please issue a refund for order A100.")
    graph.invoke(task, config=config_for("t4"))
    final = graph.invoke(Command(resume=False), config=config_for("t4"))
    result = "OK"
    if final["approved"] or "issue_refund" in tools_used(final):
        result = "WRONG"
    print(f"case 4 (rejected)        -> {result}")

def test_unexpected_route() -> None:
```

```
    state = start("anything")
    state["next_action"] = "delete_everything"
    result = "WRONG"
    try:
        route(state)
    except UnhandledRouteError:
        result = "OK"
    print(f"case 5 (unknown route)   -> {result}")

if __name__ == "__main__":
    graph = build_graph(checkpointer=MemorySaver())
    test_no_risky_action(graph)
    test_risky_action_pauses(graph)
    test_approved_resume(graph)
    test_rejected_resume(graph)
    test_unexpected_route()
```

Expected output:

```
case 1 (no risky action) -> OK
case 2 (pauses)          -> OK
case 3 (approved)        -> OK
case 4 (rejected)        -> OK
case 5 (unknown route)   -> OK
```

(With `LOG_LEVEL=WARNING` in `.env`; at `INFO` you'll also see one log line per tool call.)